

M04 / PROTECTED ANSWERS

M04 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 4

YOUR STANDALONE REVIEW BOOK 4

Open only after a saved attempt. 4

Contents 4

Open only after a genuine attempt 4

Review DE04-01 | Python setup and first program 5

Review DE04-02 | Variables, data types and operators 5

Review DE04-03 | Strings and formatting 6

Review DE04-04 | Conditions and boolean logic 6

Review DE04-05 | Lists, tuples, sets and dictionaries 6

Review DE04-06 | Loops and iteration 7

Review DE04-07 | Functions and scope 7

Review DE04-08 | Modules, imports and project structure 8

Review DE04-09 | Exceptions and debugging basics 8

Review DE04-10 | Files, paths, CSV and JSON I/O 9

Review DE04-11 | pandas foundations 9

Review DE04-12 | Practical Big-O intuition 10

Review DE04-13 | Coding drills: lists and strings 11

Review DE04-14 | Coding drills: dictionaries and sets 11

Review DE04-15 | Stack, queue, search and sort basics 12

Review DE04-16 | Integrated Python checkpoint 12

Fresh checkpoint controls - open only after your changed attempt 13

Cluster Check Review Map 13

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

YOUR STANDALONE REVIEW BOOK

Open only after a saved attempt.

Use this book to diagnose a saved first attempt. Compare reasoning, observed state and controls rather than wording. Record the weakness, close this review, then solve the changed retry independently. Work done while review is open is repair evidence, not mastery evidence.

Contents

Review DE04-01 Python setup and first program	3
Review DE04-02 Variables, data types and operators	4
Review DE04-03 Strings and formatting	5
Review DE04-04 Conditions and boolean logic	6
Review DE04-05 Lists, tuples, sets and dictionaries	7
Review DE04-06 Loops and iteration	8
Review DE04-07 Functions and scope	9
Review DE04-08 Modules, imports and project structure	10
Review DE04-09 Exceptions and debugging basics	11
Review DE04-10 Files, paths, CSV and JSON I/O	12
Review DE04-11 pandas foundations	13
Review DE04-12 Practical Big-O intuition	14
Review DE04-13 Coding drills: lists and strings	15
Review DE04-14 Coding drills: dictionaries and sets	16
Review DE04-15 Stack, queue, search and sort basics	17
Review DE04-16 Integrated Python checkpoint	18

Open only after a genuine attempt

This book is not a second learner book and not a shortcut around practice. Save the matching attempt first. Review only the lesson you need, compare reasoning rather than prose, write the weakness you found, close this book, and complete the changed retry from a blank file. Work performed while this review is open is repair evidence, not mastery evidence.

Review DE04-01 | Python setup and first program

A satisfactory first attempt proves three separate things: Python is discoverable, the intended file is the file that ran, and you can use a traceback rather than randomly changing setup.

A minimal correct script could be:

```
print("Ashish")
print("Standalone DE 2026")
print(7 * 8)
```

Expected final numeric output is 56. The exact Python version is not hard-coded as a mastery answer because it depends on what is currently installed. Your evidence should contain the version you actually observed.

For a syntax failure, a good explanation names what Python could not parse, for example: "I left a string quote open, so Python could not determine where the string ended; the SyntaxError pointed near that statement." A weak explanation is only "there was red text."

If `py --version` works but `py file.py` cannot find the file, reinstalling Python is not the next step. Verify the terminal directory and filename. If the editor runs but terminal does not, compare interpreter/path context.

Repair test: create a new file calculating $9 * 6$, deliberately introduce a different syntax error, save the traceback, repair it, then rerun after reopening the terminal. Expected arithmetic result is 54.

Review DE04-02 | Variables, data types and operators

A strong solution preserves business meaning:

```
order_id = "03107"
qty = 4
unit_price = 850.0
status = "Completed"

gross_sales = qty * unit_price
is_large = gross_sales >= 3000
```

Expected `gross_sales` is 3400.0 and `is_large` is True. `order_id` is a string because it is an identifier, not a quantity. The fact that an identifier contains digits never proves it should be numeric.

The key conceptual repair is `=` versus `==`: assignment binds a name; comparison returns a boolean. Also remember that type affects operations: `"4" * 3` repeats text rather than doing numeric multiplication.

Changed shipment retry:

```
shipment_id = "S009"
cartons = 7
unit_cost = 425.5
delivered = False
cost = cartons * unit_cost
needs_review = cost > 2500 and not delivered
```

Expected cost is 2978.5; `needs_review` is True. If `delivered` were the string "False", not delivered would be false because a non-empty string is truthy. That is why the field contract matters.

Review DE04-03 | Strings and formatting

A good solution keeps raw and transformed text separately:

```
raw_label = "  NORTH Hub  "
clean_label = raw_label.strip().lower()
shipment_id = "s009"
display_id = shipment_id.upper()
message = f"raw={raw_label!r}; cleaned={clean_label}; id={display_id}"
```

The important idea is not the exact method chain. It is that cleaning creates a derived representation and the raw source remains available. `repr(raw_label)` makes surrounding spaces visible.

Do not call two labels identical merely because `.strip().lower()` makes them equal. That proves equality under your formatting rule, not authoritative entity identity.

Changed retry: " Pen Blue " and "PEN BLUE" both become pen blue under strip+lower, but "pen blue" keeps two internal spaces. Do not add `replace(" ", " ")` merely to force a match unless a business rule defines internal whitespace normalization.

Review DE04-04 | Conditions and boolean logic

For the rule "priority only if Completed and amount $\geq$ 2500," this is sufficient:

```
def classify(status, amount):
    if status == "Completed" and amount >= 2500:
        return "priority"
    return "standard"
```

Expected boundary cases:

Input	Expected
Completed, 2499	standard
Completed, 2500	priority
Completed, 2501	priority
Pending, 3000	standard

Replacing `and` with `or` is a logic defect because a high Pending order would pass. Replacing `>=` with `>` is a boundary defect because exactly 2500 would fail.

For three-state data, do not silently collapse `None` into `False` when unknown needs separate treatment. The changed shipment retry should escalate 2+ delayed days only when `priority_customer` is `True`; define what `None` means rather than letting truthiness decide invisibly.

Review DE04-05 | Lists, tuples, sets and dictionaries

One acceptable representation:

```
steps = ["extract", "validate", "publish"]
period = (2026, 9)
failed = {"B01", "B02", "B01"}
manager = {"East": "Asha", "West": "Om"}
```

Expected failed set contains two unique IDs. Set output order is not a contract you should depend on. A dict is right only if one value per key represents the relationship you need.

To demonstrate shared mutable state:

```
a = ["extract"]
b = a
b.append("load")
assert a == ["extract", "load"]
```

`b = a` creates another reference to the same list. `a.copy()` creates another outer list, but nested mutable elements still require deeper reasoning.

For a zone with two supervisors, avoid overwriting:

```
zone_supervisors = {"Z1": ["Asha", "Ravi"]}
```

The exact structure can differ if the requirements differ; the important repair is to represent one-to-many deliberately.

Review DE04-06 | Loops and iteration

For `amounts=[1200,500,0,800,1500]`, accepted total is 4000, positive count 4 and rejected list `[0]`.

```
amounts = [1200, 500, 0, 800, 1500]
original = amounts.copy()
total = 0
positive_count = 0
invalid = []
for amount in amounts:
    if amount <= 0:
        invalid.append(amount)
        continue
    total += amount
    positive_count += 1
assert amounts == original
```

Empty input should naturally produce total 0, count 0 and empty rejected list if those are your declared semantics.

The important repair is avoiding deletion from the same list being iterated. Index positions shift after removal, so elements can be skipped. Build a result collection instead unless in-place mutation is explicitly designed and tested.

Review DE04-07 | Functions and scope

An acceptable function:

```
def summarize_order(order_id, qty, unit_price):
    return {
        "order_id": order_id,
        "gross_sales": qty * unit_price,
    }
```

Test examples should include multiple inputs; zero quantity should return gross 0 rather than fail. A function that only prints the answer returns `None` unless it explicitly returns another value.

A good teach-back says: parameters expose dependencies, local variables belong to the call's scope, and return lets later code use/test the result. Hidden globals make behavior less visible and can create accidental coupling.

Changed retry:

```
def shipment_cost(shipment_id, cartons, cost_per_carton, currency="INR"):
    return {
        "shipment_id": shipment_id,
        "cost": cartons * cost_per_carton,
        "currency": currency,
    }
```

The currency default is immutable text and is safe as a simple default. Do not use a mutable list/dict default for accumulating state.

Review DE04-08 | Modules, imports and project structure

A clean beginner structure has reusable helpers in one module and run orchestration in another:

```
# run_order.py
from stage04_helpers import gross_sales, normalized_label

def main():
    print(normalized_label(" EAST HUB "), gross_sales(3, 500.0))

if __name__ == "__main__":
    main()
```

Importing a module executes its top-level statements. Therefore a helper file that prints, deletes files or starts a pipeline at top level has import side effects. The `__main__` guard prevents the entry script's `main()` from running merely because another module imports it.

If import csv unexpectedly loads your own csv.py, you created a shadowing problem. Rename the local file, remove stale cache if necessary, and inspect imported module paths rather than reinstalling Python.

Changed retry should split `shipment_helpers.py` from `run_shipment.py` and prove importing the helper produces no workflow output.

Review DE04-09 | Exceptions and debugging basics

The supplied debugging file is intentionally designed to expose more than one assumption. A strong repair does not jump straight to a broad `try/except`.

For the record with `qty="3"` and `unit_price=500.0`, multiplication by a float fails because the quantity is text. If the contract permits integer text, parse explicitly:

```
qty = int(order["qty"])
sales = qty * order["unit_price"]
```

Then validate `qty > 0` separately. For the zero-quantity row, calculating `sales / qty` is undefined and raises `ZeroDivisionError`. Returning 0 automatically would invent a business meaning. A deliberate policy might reject the record or skip the average while preserving the reason.

A useful narrow validator:

```
def safe_positive_int(raw_value):
    try:
```

```

    value = int(raw_value)
except (TypeError, ValueError) as exc:
    raise ValueError(f"not an integer: {raw_value!r}") from exc
if value <= 0:
    raise ValueError(f"must be positive: {value}")
return value

```

Expected: "4" -> 4; "0", "-2" and "four" raise ValueError. This distinguishes parse failure from business-rule failure while exposing both through a clear interface.

Review DE04-10 | Files, paths, CSV and JSON I/O

The retail file contains 25 data rows and the required header fields. An acceptable boundary check uses `csv.DictReader`, obtains fieldnames, computes missing required columns, and stops before transformation if the schema is incomplete.

A reproducible path should be based on the project/script location or an explicit command-line/config input - not `C:\Users\....`

Minimal manifest logic:

```

manifest = {
    "source": orders_path.name,
    "expected_rows": len(rows),
    "status": "training",
}
with manifest_path.open("w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2, sort_keys=True)
with manifest_path.open("r", encoding="utf-8") as f:
    loaded = json.load(f)
assert loaded == manifest

```

M04_Bad_Orders.csv is missing qty. The best first failure is a clear missing-column validation, not a later calculation error. On the shipment retry the required fields are `shipment_id`, `hub`, `status`, `cartons`, `unit_cost`, `coordinator`; if you deliberately require a nonexistent field, your validation should name it.

Review DE04-11 | pandas foundations

The canonical retail controls are:

Check	Expected
input rows	25
Completed rows	23
Completed sales	39320
missing sales_rep	1
East Completed sales	11700
North Completed sales	9310
South Completed sales	10210
West Completed sales	8100

One acceptable solution shape:

```
df = pd.read_csv(path)
assert len(df) == 25
assert df["order_id"].is_unique

df["sales"] = df["qty"] * df["unit_price"]
completed = df.loc[df["status"].eq("Completed")].copy()
regional = completed.groupby("region", as_index=False, dropna=False)["sales"].sum()
assert completed["sales"].sum() == regional["sales"].sum() == 39320

top3 = completed.sort_values(
    ["sales", "order_id"], ascending=[False, True]
).head(3)
missing = df["sales_rep"].isna().sum()
```

State grain: detail output = one Completed order; regional output = one region; missing-rep rows = one source order per returned row; top3 = one Completed order per returned row.

Shipment changed-retry controls, reveal only after attempting:

Check	Expected
input rows	16
Delivered rows	13
Delivered shipment value	19450

missing coordinator over all input rows

Patna Delivered value 4000

Check	Expected
Ranchi Delivered value	4900
Kolkata Delivered value	5850
Delhi Delivered value	4700

If your grouped total differs from 19450, compare source row count -> Delivered row count -> calculated shipment_value -> grouped total. Do not add fillna/dropna merely to force the expected number.

Review DE04-12 | Practical Big-O intuition

The core reasoning is about growth:

- one scan over n records -> $O(n)$;
- each of n records scanning n others -> $O(n^2)$;
- building a set/dict from n records -> approximately $O(n)$ average construction, then average
- $O(1)$ membership/lookup per key;
- comparison sorting -> commonly $O(n \log n)$.

For "each shipment scans every failed ID," if there are n shipments and roughly n failed IDs, repeated scanning can approach $O(n^2)$. Building a failed-ID set is $O(n)$ average setup and n average $O(1)$ membership checks, yielding roughly $O(n)$ overall average-growth intuition for that two-step pattern.

Do not say sets are always faster for every tiny input. Do not use wall-clock time from four elements as proof of asymptotic complexity. Also do not claim dict/set lookup is mathematically guaranteed $O(1)$ in every pathological case; "average $O(1)$ " is the intended foundation statement.

Customer-map retry: build customer_id -> region once, then retrieve by key for each order. The key insight is eliminating repeated full scans.

Review DE04-13 | Coding drills: lists and strings

One acceptable reverse_words:

```
def reverse_words(text):
    return " ".join(text.split()[::-1])
```

One acceptable count_vowels:

```
def count_vowels(text):
    vowels = {"a", "e", "i", "o", "u"}
    return sum(1 for char in text.lower() if char in vowels)
```

These compact forms are not inherently better than the more explicit guided versions. You must be able to explain input -> intermediate -> output and edge cases.

Changed retry:

```
def normalize_words(text):
    return text.strip().lower().split()
```

Because `.split()` with no argument already handles runs of whitespace, blank/whitespace-only text returns `[]`. Confirm with tests rather than assuming.

Review DE04-14 | Coding drills: dictionaries and sets

One acceptable frequency map:

```
def frequency_map(items):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts
```

One acceptable unique-in-order:

```
def unique_in_order(items):
    seen = set()
    result = []
    for item in items:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
```

For the changed composite-key problem, count (order_id, line_no) tuples, then return keys whose count exceeds one. Checking order_id alone is insufficient because several legitimate lines can share one order ID.

A robust implementation can preserve first duplicate detection order without emitting the same duplicate key twice by using a second reported set or by building counts first and selecting unique keys later.

Review DE04-15 | Stack, queue, search and sort basics

Expected stack behavior for extract, transform, load: pop returns load. Expected queue behavior: popleft returns extract.

Linear search:

```
def linear_search(values, target):
    for i, value in enumerate(values):
        if value == target:
            return i
    return -1
```

Binary search requires ascending sorted input:

```
def binary_search(values, target):
    low, high = 0, len(values) - 1
    while low <= high:
        mid = (low + high) // 2
        if values[mid] == target:
            return mid
        if values[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

For [1,3,4,7,12], target 7 is at index 3. For target 6 return -1.

sorted([7,2,9,1,5]) returns [1,2,5,7,9] while preserving the original list. .sort() mutates and returns None. If your answer says binary search works on [4,12,7,3] directly, the core precondition is missing.

Review DE04-16 | Integrated Python checkpoint

Do not use this review to copy a finished program. Compare your design against the control contract and repair only weak boundaries.

A valid solution should separate at least these concerns conceptually, whether or not you use separate functions:

```
resolve input path
-> verify file exists
-> read CSV
-> verify required columns
-> parse/validate qty + unit_price
-> build calculated rows
-> filter Completed
-> regional aggregation + top3 + missing count
-> reconcile controls
-> write deterministic JSON
```

-> read back + assert critical values

Canonical retail controls:

source rows	25
Completed rows	23
Completed sales	39320
missing sales_rep	1
East	11700
North	9310
South	10210
West	8100

The regional numbers sum to 39320. Your program should calculate them from input, not embed them as results.

A useful implementation pattern is to keep numeric parsing in one boundary function and to return clean validated records, but the exact function names are your choice. Reject a missing required column before trying to transform the row. Preserve the bad-file traceback/message as failure evidence.

For deterministic top 3, sort by a declared pair such as descending sales then ascending order_id. Do not rely on incidental input order for ties.

The output JSON should contain enough source/run metadata for another person to understand what was processed. Read it back and assert critical values. A successful write syscall does not prove the file contains the intended logical output.

Fresh checkpoint controls - open only after your changed attempt

The shipment retest controls are:

source rows	16
Delivered rows	13
Delivered shipment value	19450
missing coordinator	1 (over all input rows)
Patna	4000
Ranchi	4900
Kolkata	5850
Delhi	4700

These totals must be recalculated from M04_Retest_Shipments.csv. The fresh task changes domain, field names and report contract while testing the same Python foundations. If you simply rename variables in the retail solution without re-reading the new schema/population, that is not independent transfer.

Cluster Check Review Map

For Cluster 1, weakness in values/types -> DE04-02; text transformation -> DE04-03; branching/boundaries -> DE04-04.

For Cluster 2, structure choice -> DE04-05; iteration/state -> DE04-06; functions/return -> DE04-07; import/entry point -> DE04-08.

For Cluster 3, exception/debugging -> DE04-09; file/schema boundary -> DE04-10; pandas grain/reconciliation -> DE04-11; complexity reasoning -> DE04-12.

For Cluster 4, list/string coding -> DE04-13; dict/set coding -> DE04-14; stack/queue/search/sort -> DE04-15.

A weak cluster check does not mean restart M04. Repair the exact competency, close review, complete a changed micro-task, then repeat a fresh check.